

Smart Text Analyzer - Local LLM Application

Subject: Speech & Language Processing **Student:** Irene Valle

Date: April 2026

Folder: Lab 3

Problem Description

For this lab, I developed a small application called **Smart Text Analyzer**. The program allows the user to enter a piece of text—such as a product review, an article excerpt, or a comment—and automatically obtain:

- Sentiment analysis (positive, negative, or neutral)
- Keyword extraction
- A short summary of the text

The objective of the application is not only to query a local LLM, but to build a complete NLP pipeline around it. For that reason, the system includes preprocessing, prompt engineering, postprocessing, and a graphical user interface.

System Design and Workflow

The system follows these steps:

1. The user enters a text and selects a local model in the Streamlit interface.
2. The text is preprocessed with NLTK by converting it to lowercase, removing punctuation, and filtering stopwords.
3. A structured prompt is created to force the LLM to return the analysis in JSON format.
4. The selected local model is called through Ollama.
5. The output is postprocessed: the JSON is extracted, parsed, and the results are displayed in the interface.

Workflow Diagram

User Input

↓

Preprocessing (NLTK)

↓

Structured Prompt

↓

Ollama (Local LLM)

↓

JSON Parsing + Postprocessing

↓

Results: Sentiment + Keywords + Summary

Model Selection and Justification

Ollama was used to run the language model locally.

The main model selected was **llama3.2:1b** because:

- It is lightweight and fast on standard computers
- It requires relatively few computational resources
- It is sufficient for basic English text analysis tasks
- It allows the application to run fully locally

The intended model for this project is **llama3.2:1b**, executed locally through Ollama. An alternative lightweight model was considered during development, but it was not installed in the final environment. Due to time and network constraints during the final setup, the download of the local model through Ollama was still in progress at submission time. However, the application code, interface, workflow, and technical design are complete and ready to run once the model is fully downloaded.

Implementation Details

Tools Used

- Python 3
- Streamlit, for the graphical user interface
- Ollama (Python library), to communicate with the local LLM
- NLTK, for stopwords and basic text preprocessing
- Standard Python libraries: **re**, **json**, and **time**

Design Decisions

- The code was kept simple and readable.
- A strict prompt was used to request a structured JSON output.
- Basic preprocessing was added to reduce noise before sending the text to the model.
- Postprocessing was included to extract and parse the JSON safely.
- Response time measurement was added to provide feedback about model speed.

Main Code File

local_llm_text_analyzer.py

The complete implementation is included in the corresponding file in this folder.

Results and Screenshots

The application interface, processing pipeline, and integration with Ollama were completed. Final runtime validation depends on the successful completion of the local model download.

The report includes screenshots of:

- The main interface before entering text [Main interface](#)
- An example with text entered and a model selected [Example](#)
- The final results displayed in the interface [Results](#)

Discussion, Limitations, and Possible Improvements

This project was designed as a small but complete NLP system built around a local LLM. Instead of using a simple prompt-response interaction, the application integrates preprocessing, prompt engineering, JSON-based structured output, response parsing, and visual presentation of results.

Limitations

Some limitations were observed during testing:

- A practical limitation of local LLM deployment is the initial model download time, especially under slow network conditions.
- Small local models such as `llama3.2:1b` may occasionally hallucinate or return invalid JSON
- The quality of sentiment analysis and summarization is acceptable, but lower than with larger models
- Response time depends on the selected model and the length of the input text
- The preprocessing pipeline is limited to English stopwords and basic cleaning

Possible Improvements

Some possible future improvements are:

- Testing stronger local models such as `llama3.2:3b` or `llama3.1:8b`
- Improving prompt design to obtain more stable structured outputs
- Adding output validation or retry logic when JSON is malformed
- Saving previous analyses or adding a history panel in the interface
- Adding visualizations such as a keyword cloud
- Extending the system with embeddings or retrieval for a more advanced project

How to Run the Application

```
# 1. Install dependencies
pip install -r requirements.txt

# 2. Download the model (only the first time)
ollama pull llama3.2:1b

# 3. Run the application
streamlit run local_llm_text_analyzer.py
```